HARNESS LAB · MÓDULO 5 · AULA 5.2

Condições de parada: *o freio do loop*

Um loop sem freio é o que estoura a conta. Três tentativas, teto de gasto e de tempo por sessão, e uma saída honesta: quando não fecha, o loop para e chama você.

`MAX_CICLOS=3` `--max-cycles` `--max-budget-usd` `timeout` `.claude/scripts/shim/claude` `PATH` `gate 0` `loop.sh --checa` `Ctrl-C`

VOCÊ SAI COM

O shim e o `loop.sh` com freio commitados no seu repositório: três ciclos por fase, teto de gasto e de tempo em toda sessão, um pré-voo que barra antes de gastar, e o freio puxado no motor falso, sem gastar token.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

O loop da 5.1 já roda. Esta aula põe o **freio de mão**, uma contenção de danos para o caso absurdo: teste que falha sempre, juiz duro demais, loop infinito. O que falha sempre não passa para a frente e precisa de um humano olhando. O Lucas conta que isso nunca aconteceu com ele: o trilho, os hooks e a validação das aulas anteriores deixam o loop estável.

Máximo de ciclos

Falhou nos testes ou no juiz: tenta de novo. Esgotou, para e um humano decide. Recomendação: **três tentativas**.

Teto de gasto

Plano enxuto? Em vez de ciclos, trava pelo consumo: um máximo de budget **por funcionalidade**.

Turnos, ciclos, consumo

A ferramenta entrega os três: quando a sessão termina, traz o que gastou.

No Claude Code, o resultado da sessão headless traz o gasto estimado (`total_cost_usd`) e os turnos (`num_turns`), e as flags `--max-turns` e `--max-budget-usd` cortam a sessão. O exemplo da aula é o **shim** `claude`, um arquivo na frente do `PATH`: toda sessão do loop passa por ele e ganha teto de gasto e de tempo, sem editar o ralph. Ganhar freio não é ruim: dificilmente você vai precisar, mas é uma segurança a mais.



Vale para as duas sessões de cada ciclo: a implementação e o juiz. O Ctrl-C atravessa o shim e para tudo.

Saída honesta

Insistindo, uma hora o modelo faz qualquer coisa que passe no arcabouço, como remover o teste para passar no teste (a deriva de instrução da 1.3). A saída honesta é parar e chamar o humano.

Os sinais de que travou, e quem pega cada um:

SINAL	QUEM PEGA
Sessão terminou sem resultado	Gate 0 do ralph: o ciclo reprova
Sessão não escreveu nada	Gate 1 avisa; o juiz reprova ("não tem nada feito")
Sessão que não termina	O `timeout` do seu shim corta
Log parado, processo morto	O vigia da aula 5.3, de fora
Edição no teste de aceite	O hook `guarda-aceite.py` (Módulo 4), exigido no pré-voo
Muito ciclo no mesmo arquivo	A sua revisão (Módulo 6)

Os dois últimos sinais vêm da lista do Claude 3.7 Sonnet System Card (seção 6, p. 23).

> *"What stops it? `--max-turns`, `--max-budget-usd`, a spend limit." "Give it a cap on turns or spend, and have a person approve the result at the end."*
>
> Documentação oficial do Claude Code · Best practices, "Let Claude run without you" (code.claude.com/docs/en/best-practices)

A recomendação da aula: usar o shim para os freios. Com isso fecha o arcabouço do loop: freios, condição de parada, avaliação e iteração rodando juntos.

2 Indo *além da aula*

ALÉM DA AULA · O TETO É POR SESSÃO; O DA FUNCIONALIDADE SAI DA CONTA

O `--max-budget-usd` vale para uma sessão `claude -p`; cada ciclo abre sessões novas, com teto novo (a implementação e, se a Prova passou, o juiz). O teto da feature é a conta: **fases x ciclos x 2 sessões** (o `--checa` imprime) vezes o teto, 18 x US$ 5 = US$ 90 no exemplo da seção 3, teórico. Então o teto por sessão fica **acima da sessão mais cara que você já viu**, com folga; abaixo de uma sessão normal, ele corta a sessão no meio e gasta um ciclo, até a fase falhar. Conta alta pede fase menor ou menos ciclos. Sessões de implementação de runs reais do Lucas:

RUN	US$ POR SESSÃO	TURNOS	MINUTOS
4 fases, 3 já feitas (19/08)	0,58 a 2,19	7 a 43	1 a 5
Outros dois (01/09 e 28/10)	0,94 a 4,39	21 a 65	3 a 14
12 fases (15/09)	0,72 a 13,89	9 a 116	1 a 24
8 fases (06/10)	0,46 a 15,51	11 a 122	3 a 95

Os padrões (US$ 5 e 40m) cortariam 10 de 15 sessões do run de 12 fases e 6 de 10 do de 8.

Fonte: cli-reference; `ralph.sh` 0.2.0; linha de resultado de cada log de sessão (`total_cost_usd`, estimativa do cliente). Sessão repetida após espera de limite fica fora da conta.

ALÉM DA AULA · TURNOS E GASTO: ONDE LER, E POR QUE O SHIM NÃO CORTA POR TURNO

A sessão de implementação termina com a linha de resultado que o gate 0 procura: `num_turns`, `total_cost_usd` e `duration_ms` (o juiz roda em modo texto e não registra). O leitor está no lab, passo 1. O shim não põe `--max-turns`: turno não mede dinheiro nem progresso (7 a 122 por sessão na tabela), e cortaria também o juiz, que passa pelo shim. Se quiser, na sua cópia do shim, acrescente `--max-turns "${LOOP_MAX_TURNS:-200}"` (exemplo, acima da sua sessão mais longa); o `instalar.sh` das aulas seguintes vai dar conflito nesse arquivo. A sessão cortada termina com `error_max_turns` e vira ciclo de correção.

Fonte: cli-reference, `--max-turns`; agent-sdk/agent-loop, "Turns and budget" e "Handle the result".

ALÉM DA AULA · QUEM PARA NO GASTO É O CLAUDE CODE, E ELE PASSA UM POUCO

O shim só acrescenta a flag. Quem soma e corta é o CLI, pela **estimativa do cliente**, conferida depois de cada resposta: a que cruza o teto termina e conta, por isso a doc manda deixar folga. A sessão cortada sai com `error_max_budget_usd`, e os gates decidem como em qualquer outra. Na assinatura, o gasto conta no limite de uso; com credencial de API no ambiente, vira fatura (o `--checa` recusa, passo 6). O `--max-budget-usd` no shim ainda não rodou com o CLI real: o primeiro uso é o ensaio da 5.3.

Fonte: cli-reference ("Spend can pass the cap, so leave headroom"); agent-sdk/agent-loop, "Budget headroom"; authentication, "Authentication precedence".

ALÉM DA AULA · O ESTOURO DE TEMPO E O CTRL-C PASSAM PELO MESMO TRAP

Estourou o `LOOP_TIMEOUT`: o `timeout` manda TERM à sessão (KILL 60 s depois: o `-k 60`) e sai com 124. No SIGTERM, o `claude -p` encerra o Bash em curso e **não registra resultado**: o gate 0 reprova e o ciclo seguinte recebe "O engine terminou sem emitir um resultado". Sem `--foreground`, o `timeout` fica num grupo de processos próprio e não recebe o Ctrl-C. Com `exec timeout ...`, o Ctrl-C não para nada: o loop fica esperando a sessão, que segue mexendo no repositório (no teste, os dois vivos depois de 10 s). O shim da aula repassa o Ctrl-C como TERM: tudo saiu em menos de um segundo, com exit 143, não 130.

Fonte: headless, "Stop a run with SIGTERM"; `timeout --help` (GNU coreutils); `testa-loop.sh` do curso, com o shim antigo e o novo.

ALÉM DA AULA · OS GATES REPETEM O VEREDITO; O MODELO NÃO REPETE A TENTATIVA

A Prova e o juiz dão o mesmo veredito para o mesmo código. O modelo, não: cada ciclo é uma sessão nova, com a causa no prompt, e ele tenta outra coisa. Ajuda até um ponto: "After repeated failures, it sometimes implements special cases for problematic tests" (Claude 3.7 Sonnet System Card, p. 22). Nos episódios de treino do Claude Opus 5.5, diante de tarefa impossível, a tentativa de burlar sobe de três a seis vezes, e cerca de 80% delas é entregar incompleto sabendo (System Card, seção 6.2.2, p. 101): é o que o juiz reprova. E a saída honesta funciona: no ImpossibleBench, a opção de chamar um humano baixou a trapaça do GPT-5 de **54% para 9%**, e mais tentativas subiram a trapaça de 33% para 38% (no Claude Opus 4.1, efeito bem menor; arXiv 2510.20270, p. 8). Por isso três ciclos e para. O ralph para na primeira fase que falha; `--keep-going` faz um commit `wip` e segue, só com fases independentes.

ALÉM DA AULA · LIMITE SEMANAL: O RALPH NÃO RECONHECE (DEDUZIDO DO CÓDIGO)

O ralph espera o reset do limite de sessão e do de 5 horas sem gastar ciclo, mas as frases de limite semanal, de Opus e de Sonnet ("You've hit your weekly limit · resets Mon 12:00am") não casam com a regex dele. Na implementação, se o limite vier como erro 429, casa o `"api_error_status":429` do JSON de resultado, sem horário: espera padrão de 30 min. Efeito provável, nunca observado: essa espera até 20 vezes e o run aborta. No juiz, em modo texto, não há JSON: o limite passa batido e queima um ciclo. Antes de um run longo, confira o `/usage`.

Fonte: errors, "You've hit your session limit"; teste da detecção de limite do `ralph.sh` 0.2.0 com as frases da doc.

3 Instale e faça *no seu repositório*

A aula instala em `.claude/scripts/` os mesmos cinco scripts da 5.1 e da 5.3 (`loop.sh`, `confere-phases.sh`, `aviso.sh`, `vigia.sh`, `shim/claude`): quem já instalou a 5.1 vê "igual". Antes: bc-harness (aula 3.4), hook `guarda-aceite.py` (`instalar.sh 4.2`) com o `.claude/aceite.txt`, o PHASES do `/plan` da 4.3 (ou `/bc-harness:plan`, sem a cópia local), GNU `timeout` e bash 4+. Para o lab, também `python3`, `setsid` (util-linux; já vêm no Ubuntu e no WSL) e `jq`. Linux ou WSL; macOS não testado.

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou antes? Atualize:

Terminal · em qualquer pasta — shell

```
git -C ~/harness-lab-materiais pull
```

2 Confira o bc-harness: o motor do loop, o `ralph.sh`, vem dele.

Se o `claude plugin list` já mostra `bc-harness@beer-and-code`, pule.

Terminal — claude

```
claude plugin marketplace add beerandcodeteam/beer-and-code-harness
claude plugin install bc-harness@beer-and-code
claude plugin list
```

3 Fique numa branch de trabalho.

O loop commita sozinho e o pré-voo recusa `main` e `master`. Já está na branch da feature? Pule o segundo.

Terminal · na raiz do seu projeto — shell

```
git branch --show-current
git switch -c feat/feature-alvo
```

4 Instale os scripts e commite.

O pré-voo exige árvore limpa, porque o ralph commita com `git add -A`. Se o `instalar.sh` disser "conflito" em `loop.sh` (você escreveu o seu na 5.1), pare antes do commit: tire o `loop.sh.harness-lab-novo` do projeto e siga o `modelos/pedido-freios.md` (fim desta seção).

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 5.2
```

Terminal · na raiz do seu projeto — shell

```
git add .claude/scripts
git diff --cached --quiet || git commit -m "harness: freios do loop"
```

5 Rode o pré-voo. Zero token: nada é chamado.

O primeiro comando pega o PHASES mais recente de `.spec/features/` e o `echo` mostra qual. Tem mais de uma feature? Troque pelo caminho do seu.

Terminal · na raiz do seu projeto — shell

```
P=$(ls -t .spec/features/*/PHASES.md | head -1)
echo "$P"
.claude/scripts/loop.sh --checa "$P"
```

Esperado: só linhas `ok`, as camadas do harness e "pré-voo ok". Qualquer `NÃO` barra o run; a seção 5 diz o que fazer com cada um.

saída do `--checa` no projeto de teste (trecho) — o que você deve ver

```
  ok   Prova: php artisan test (detectado)
  ok   confere-phases: fases: 3 · tasks: 7 · erros: 0 · avisos: 0
  ok   freios: MAX_CICLOS=3, LOOP_TIMEOUT=40m, LOOP_BUDGET_USD=5 por sessão
(...)
  pior caso: 3 fase(s) x 3 ciclos x 2 sessões = 18 sessões
pré-voo ok. Para rodar: .claude/scripts/loop.sh .spec/features/agenda-ics/PHASES.md
```

6 Prove que a credencial de API barra o run.

A chave é falsa e só existe neste comando. Esperado: "NÃO credencial de API no ambiente: ANTHROPIC_API_KEY" e "Nada rodou, nenhum token gasto".

Terminal · na raiz do seu projeto — shell

```
P=$(ls -t .spec/features/*/PHASES.md | head -1)
ANTHROPIC_API_KEY=teste .claude/scripts/loop.sh --checa "$P" || true
```

7 Escolha os seus números e veja a conta mudar.

Variáveis de ambiente, valendo só para aquele comando. Os números são exemplo, com folga sobre a sessão mais cara da seção 2 (US$ 15,51). Repare nas linhas "freios" e "pior caso".

Terminal · na raiz do seu projeto — shell

```
P=$(ls -t .spec/features/*/PHASES.md | head -1)
LOOP_BUDGET_USD=20 MAX_CICLOS=2 .claude/scripts/loop.sh --checa "$P"
```

- `LOOP_TIMEOUT` (padrão 40m): acima da sua **sessão** mais longa (o `duration_ms`; o leitor do lab mostra em minutos). Nos runs reais, de 1 a 24 min na maioria, e uma de 95. A fase (6 a 33 min na maioria) soma gates e juiz: é teto da sessão, não a medida.
- `LOOP_BUDGET_USD` (padrão 5): acima da sua sessão mais cara, com folga. É estimativa.
- `MAX_CICLOS` (padrão 3): tentativas antes de te chamar. A aula recomenda 3.

Os padrões cobrem sessões como as das duas primeiras linhas da tabela da seção 2. No run de verdade, é o mesmo comando sem o `--checa`: chama o `claude` e gasta (o primeiro run pago do curso é o ensaio da 5.3). Antes de um run longo, confira o `/usage`.

O shim *instalado*

Seis linhas de código. A guarda do `CLAUDE_REAL` impede chamar um comando vazio fora do `loop.sh`; o `&`, o `trap` e o `wait` deixam o Ctrl-C passar. No `loop.sh`, o freio está nos padrões (linhas 43 e 44), no pré-voo completo (125 a 185), no shim na frente do `PATH` (247 a 249) e no `--max-cycles` (258). O bloco `[5.2]` é desta aula.

.claude/scripts/shim/claude — instalado pelo instalar.sh · não copie do PDF

```
#!/usr/bin/env bash
# .claude/scripts/shim/claude · teto de tempo e de gasto por sessão, sem editar o ralph
# (...) LOOP_TIMEOUT padrão 40m; LOOP_BUDGET_USD padrão 5.
: "${CLAUDE_REAL:?CLAUDE_REAL vazio: rode pelo loop.sh, (...)}"
timeout -k 60 "${LOOP_TIMEOUT:-40m}" "$CLAUDE_REAL" --max-budget-usd "${LOOP_BUDGET_USD:-5}" "$@" &
pid=$!
trap 'kill -TERM "$pid" 2>/dev/null' INT TERM HUP
wait "$pid"; rc=$?
if [ "$rc" -gt 128 ] && kill -0 "$pid" 2>/dev/null; then wait "$pid"; rc=$?; fi
exit "$rc"
```

Puxe o freio *sem gastar token*

A pasta `lab/` da aula tem o `demo.sh`: cria um repositório de exemplo (2 fases, a Prova é um `test.sh`), põe um `claude` falso no `PATH` e roda o seu `ralph.sh` de verdade pelo `loop.sh`. No cenário `pendura`, a sessão nunca termina.

1

O estouro: 20 s por sessão, 2 ciclos. Depois, onde parou.

Cerca de 45 s. A cada ciclo, o aviso do gate 1 ("a sessao nao escreveu nada") e logo "Gate 0 vermelho", que é a ordem do código; depois "FALHOU apos 2 ciclos" e `exit: 1`. A linha "aviso: ..." é a peça da 5.3.

●●● Terminal · em qualquer pasta — shell

```
S=~/harness-lab-materiais/modulo-5-loop/aula-5.2-condicoes-de-parada/lab
export DEMO_DIR=$HOME/harness-lab-sandbox
F="MOTOR_CENARIO=pendura LOOP_TIMEOUT=20s MAX_CICLOS=2"
env $F bash $S/testes/demo.sh || true
```

O `resumo.txt` diz "desfecho: FALHOU" e "onde parou: F1 (Primeira fase), ciclo 2/2, gate 0". O prompt do ciclo 2 traz a causa: "O engine terminou sem emitir um resultado".

●●● Terminal · em qualquer pasta — shell

```
R=$(ls -td ~/harness-lab-sandbox/run.*/repo-exemplo | head -1)
cat $R/.loop-logs/run-*/resumo.txt
grep -A3 "Motivo da falha" $R/.phases/prompts/phase-01.cycle-2.txt
```

Agora os números de cada sessão. Aqui saem só os nomes dos logs: as sessões cortadas não deixaram linha de resultado.

Terminal · em qualquer pasta — shell

```
cd "$(ls -td ~/harness-lab-sandbox/run.*/repo-exemplo | head -1)"
cd "$(ls -td .loop-logs/*/logs | head -1)"
F='fromjson?|select(.type=="result")'
C='[.num_turns,.total_cost_usd,(.duration_ms/6e4|round)]'
for f in *.cycle-*.log; do echo "$f $(jq -cR "$F|$C" "$f")"; done
```

No seu projeto, depois de um run, rode da segunda linha em diante, na raiz (`cd -` volta). Cada sessão sai com turnos, US$ estimado e minutos: `phase-04.cycle-1.log [43,2.1890825,5]`.

2 **O freio de mão: Ctrl-C depois de uns 10 s.**

O loop volta na hora com "desfecho: INTERROMPIDO". O `pgrep` não pode listar nada: nenhuma sessão ficou viva.

Terminal · em qualquer pasta · aperte Ctrl-C — shell

```
S=~/harness-lab-materiais/modulo-5-loop/aula-5.2-condicoes-de-parada/lab
env MOTOR_CENARIO=pendura LOOP_TIMEOUT=5m bash $S/testes/demo.sh
pgrep -af -- --max-budget-usd
```

3 **Passe os seus scripts no teste do loop.**

Roda o seu `loop.sh` e o seu shim em repositórios temporários, com o ralph de verdade e o motor falso: estouro, Ctrl-C, pré-voo, credencial, aviso. Cerca de 30 s. No fim: `testa-loop (até a aula 5.3): 102 passaram`.

Terminal · na raiz do seu projeto — shell

```
S=~/harness-lab-materiais/modulo-5-loop/aula-5.2-condicoes-de-parada/lab
C=$PWD/.claude/scripts
SHIM=$C/shim/claude bash $S/testes/testa-loop.sh $C/loop.sh
```

Escreveu o seu próprio `loop.sh` na 5.1? Em `modelos/pedido-freios.md` estão os dois pedidos para o Claude Code (o shim, com o corpo exato, e o freio do pré-voo). Cole um de cada vez na raiz do projeto. Para conferir só até os freios, o mesmo teste aceita `--ate 5.2`: 80 verificações.

4 Arquivos que *reforçam a aula*

`.claude/scripts/shim/claude`

O freio sem mexer no motor: tempo e teto em toda sessão, inclusive a do juiz, com o Ctrl-C passando.

No repositório: modulo-5-loop/aula-5.2-condicoes-de-parada/projeto/

`.claude/scripts/loop.sh`

O pré-voo que barra antes de gastar e os três números do freio num lugar só.

No repositório: modulo-5-loop/aula-5.2-condicoes-de-parada/projeto/

`lab/testes/demo.sh + lab/motor-falso/claude`

O freio puxado na sua frente, sem token. Cenários `pendura`, `sem-resultado`, `nunca-passa` e `incompleta`.

No repositório: modulo-5-loop/aula-5.2-condicoes-de-parada/lab/

`lab/testes/testa-loop.sh`

A prova de que o seu `loop.sh` e o seu shim param: 102 verificações com o ralph de verdade. Rode de novo a cada mudança neles.

No repositório: modulo-5-loop/aula-5.2-condicoes-de-parada/lab/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
`NÃO branch main`	O loop commita sozinho	`git switch -c feat/...` (passo 3)
`NÃO árvore suja`	Scripts fora do commit	Commite os scripts (passo 4)
`NÃO hook guarda-aceite.py`	Falta o hook da 4.2	`instalar.sh 4.2` e o `.claude/aceite.txt`; sem aceite, `LOOP_SEM_ACEITE=1`, sabendo do risco
`NÃO credencial de API`	Chave exportada passa na frente da assinatura	`unset ANTHROPIC_API_KEY` (e tire do `.bashrc`), ou `LOOP_ACEITO_API=1`
`NÃO nenhum comando de teste`	Stack fora da detecção (Rails, Elixir, Gradle, Maven)	`LOOP_TEST_CMD="bundle exec rspec"` na frente
`NÃO comando 'timeout'`	macOS: é do GNU coreutils	`brew install coreutils` e o `gnubin` no `PATH` (alias não vale em script). Não testado
Ctrl-C não volta; a sessão segue	Shim com `exec timeout`: o loop espera a sessão	O shim da aula; confira com `pgrep -af -- --max-budget-usd`
Sessões cortadas; a fase esgota os ciclos	Teto de gasto ou de tempo abaixo da sua sessão normal	Leia os números (lab, passo 1) e suba os tetos

6 Você sai *com*

- [ ] O shim e o `loop.sh` com freio commitados na branch da feature (`git ls-files .claude/scripts`).
- [ ] `loop.sh --checa` no seu PHASES: "pré-voo ok", a linha dos freios e o pior caso.
- [ ] Os seus três números, acima da sua sessão mais cara e da mais longa, e o teto da funcionalidade (sessões x `LOOP_BUDGET_USD`).
- [ ] O estouro no motor falso ("FALHOU apos 2 ciclos", gate 0) e o leitor de turnos, US$ e minutos rodado, sem gastar token.
- [ ] Ctrl-C no meio do run e `pgrep -af -- --max-budget-usd` vazio.
- [ ] `testa-loop.sh` com o seu `loop.sh` e o seu shim: 102 passaram.

PROMETIDO NA AULA

- ✓ **"Três tentativas"**: `MAX_CICLOS=3` no `loop.sh` (linha 43), vira `--max-cycles` (linha 258). Página 7.
- ✓ **"Um arquivo chamado shim claude"**: `projeto/.claude/scripts/shim/claude`, instalado no passo 4 (página 5), lido na página 7.
- ✓ **"Insumo pra poder colocar esse max budget usage"**, por funcionalidade: a conta e os runs reais (página 3) e o passo 7 (página 6).
- ✓ **"Máximo de turnos, máximo de ciclos, máximo de consumo"**: a caixa "Turnos e gasto" (página 3) e o leitor de `num_turns` e `total_cost_usd` no lab (página 8).
- ✓ **"Sessão que não termina, o timeout"**: `LOOP_TIMEOUT` no shim, provado no lab (página 7).
- ✓ **"A saída honesta é: para e chama o humano"**: os sinais (página 2) e o `resumo.txt` que diz onde parou (página 7).

7 *Glossário*

Ciclo	Uma tentativa da fase: implementação, gates e, se a Prova passou, o juiz.
Shim	Executável com o nome de outro, antes no `PATH`, que chama o original.
Gate 0	Sem linha de resultado no fim da sessão (estouro, morte), o ciclo reprova.
--max-budget-usd	Teto de gasto estimado de uma sessão `claude -p`. Pode passar um pouco.
Motor falso	Um `claude` de mentira, com cenários, para testar o loop sem token.
Saída honesta	Parar e chamar um humano em vez de insistir até achar um atalho.

8 Para *ir além*

- CLI reference: `--max-budget-usd` (estimativa, deixe folga) e `--max-turns`.
- How the agent loop works: "Budget headroom" e os `subtype` do resultado.
- Run Claude Code programmatically: "Stop a run with SIGTERM" e o `total_cost_usd` da saída JSON.
- Authentication: "Authentication precedence", a ordem das credenciais que decide quem paga.
- ImpossibleBench: tentativas, opção de chamar humano e trapaça em tarefa impossível.

Próxima: *Aula 5.3 · Avisa quando termina ou trava*

Ele para sozinho. E quem te conta que parou? O aviso sai do fim do `loop.sh` e chega no seu celular.